Deployment Stamps

grow by adding copies, not by growing the copy

Watch the 4-min video

In one line: run many identical, independent **stamps** (scale units, cells), each a full copy of the application and its data serving a subset of tenants, with a small **control plane** that places tenants and a **global router** that sends each tenant's traffic to its stamp.

1 One deployment, one blast radius

Your SaaS runs as one huge deployment serving 10,000 tenants. One bad configuration change takes it down, and every tenant is offline together. Growth means making the biggest database bigger, until there is no bigger instance to buy. Large SaaS providers avoid both problems the same way: they stop scaling the deployment **up** and start scaling it **out** as whole copies. Azure calls this the Deployment Stamps pattern; AWS calls it cell-based architecture.

2 What a stamp is

A stamp is a complete, independent copy of the application and its data: web tier, services, databases, caches, queues, storage. It serves a subset of tenants, perhaps a few hundred, and has a **known, tested maximum capacity**, found by load testing one stamp. To grow, you don't make a stamp bigger; you deploy another one from the same template (infrastructure as code, lesson 61). Every stamp runs the same code and configuration; a "special" hand-modified stamp defeats the pattern.

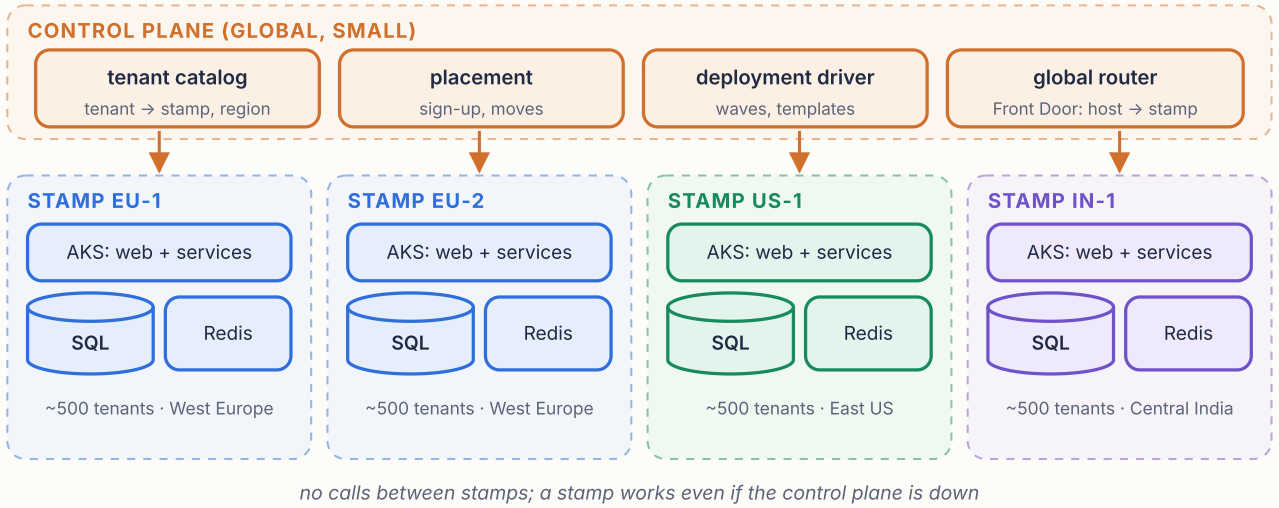


Figure 1. Control plane above, identical stamps below. The router sends each tenant to the stamp the catalog names.

The rule that makes stamps work: **no synchronous calls between stamps**, and nothing in the request path that is shared by all stamps. A shared global database or cache would put the whole fleet back into one blast radius.

3 Limiting the blast radius

With 10,000 tenants in 20 stamps of 500, a stamp failure, a corrupted database or a noisy tenant (lesson 44) affects **5%** of tenants, not 100%. Releases roll out stamp by stamp in **waves**, starting with an internal

or canary stamp, with bake time and automated health gates in between. This mirrors how the cloud providers deploy their own services region by region.

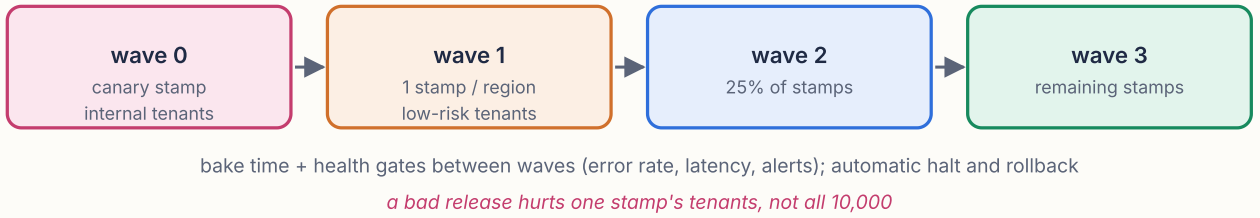


Figure 2. Safe deployment across stamps. Each wave widens exposure only after the previous one proves healthy.

Schema changes must be backward compatible (expand, then contract), because during a rollout some stamps run version N and others N+1, and a rollback must work on a stamp that already migrated.

4 The control plane and global routing

Above the stamps sits a small **control plane**: the tenant catalog (tenant → stamp, region, status), sign-up and placement, stamp inventory and capacity, and the deployment driver. A **global routing layer** such as Azure Front Door or a DNS-plus-gateway setup maps each request to its stamp, typically by host name (`acme.app.com` → stamp EU-1) or by a tenant claim in the token, with the catalog cached at the edge.

Design the control plane for **static stability**: if it goes down, existing tenants keep working, because routing uses its cached catalog and stamps don't depend on the control plane per request. Only sign-ups, moves and deployments pause. The control plane is the one global component, so keep it small, replicated and off the hot path.

5 Tenant placement

Input	Effect on placement
Residency (lesson 46)	only stamps in the tenant's allowed jurisdiction
Size and growth	big tenants spread across stamps; very large ones get a dedicated stamp
Compliance / tier	regulated or enterprise tenants on isolated stamps (lesson 41)
Stamp capacity	fill to a target (e.g. 70% of tested max), then open a new stamp
Release risk	internal and friendly tenants on the canary stamp

Placement is not one-time. Tenants grow, so rebalance by moving them between stamps with the copy + change-data-capture + catalog-flip process from lesson 42. Keep tenant data self-contained (tenant_id everywhere, globally unique IDs) so a move is routine.

6 Regional failover

Each stamp can have a **paired stamp** in another region of the same jurisdiction, with data replicated asynchronously (so a few seconds of RPO). If a region fails, the control plane promotes the paired stamps and updates routing tenant by tenant, never moving a tenant outside its allowed jurisdiction. Rehearse it: a failover runbook that has never run is a hope, not a plan.

WORKED EXAMPLE: THE AZURE DEPLOYMENT STAMPS PATTERN FOR SAAS

A global control plane holds the tenant catalog. Azure Front Door routes `{tenant}.app.com` to stamps in West Europe, East US and Central India. Each stamp is one Bicep or Terraform template: an AKS cluster, Azure SQL databases (elastic pools for small tenants), Azure Cache for Redis, Service Bus and storage. Load tests showed one stamp handles about 600 average tenants, so placement fills stamps to 500. New releases go to a canary stamp of internal tenants, then one stamp per region, then the rest in waves. When West Europe's stamps reach 70% of target, the platform team deploys EU-3 from the same template and sends new EU sign-ups there.

TRADE-OFFS

- Fixed overhead per stamp (clusters, databases, monitoring) makes small stamps inefficient: pick a size that amortises it.
- Cross-tenant features (global search, sharing between tenants on different stamps) become hard.
- Fleet operations (deploys, migrations, dashboards) must be automated; you run N systems.
- In return: linear scaling, bounded blast radius, safe rollouts, and residency almost for free.

COMMON MISTAKES

- A shared global database or cache in the request path of every stamp.
- Stamps drifting apart through manual changes.
- Deploying to all stamps at once, which throws away the blast-radius benefit.
- A control plane that every request depends on.

7 Key terms

Deployment stamp / cell: independent full copy serving some tenants

Control plane: catalog, placement and deployment above the stamps

Static stability: keeps working when the control plane fails

Scale unit: a stamp with a known, tested capacity

Global router: sends each tenant to its stamp

Deployment wave: a step of a staged rollout across stamps

8 Interview questions

Q1. What is the deployment stamps pattern? BASIC

Many identical, independent copies of the full application, each serving a subset of tenants, with a control plane that places tenants and a router that sends traffic to their stamp.

Q2. Why use it? BASIC

Scale by adding stamps, limit the blast radius of failures and deployments, and support data residency and regional failover.

Q3. A stamp is at capacity and one tenant keeps growing. What do you do? SCENARIO

Open a new stamp from the template for new sign-ups and move the growing tenant (or some neighbours) with copy + CDC + catalog flip; a very large tenant may get a dedicated stamp.

Q4. How do you roll out a risky release across 40 stamps? SCENARIO

Canary stamp first, then one stamp per region, then larger waves, with bake time and automatic health gates and rollback between waves.

Q5. What happens to tenants if the control plane is down? TRICKY

Nothing visible if it's statically stable: routing uses its cached catalog and stamps serve traffic on their own; only sign-ups, moves and deploys wait.

Q6. Why must stamps not call each other synchronously? TRICKY

Cross-stamp calls couple their availability, so one stamp's failure spreads and the blast radius is no longer one stamp.

Q7. How is a stamp different from a region? TRICKY

A region is the provider's location; a stamp is your unit of deployment. A region can host several stamps, and each stamp usually lives in one region.

Next: SD-V-001 · Cloud Service Models backendroadmap.com